SESSION 21 CAPSTONE

ServiceDesk

A help desk ticketing platform, shipped through a complete DevOps pipeline

Aman Yadav · 24bcs10183 · 2nd Year · Batch A

What the application does

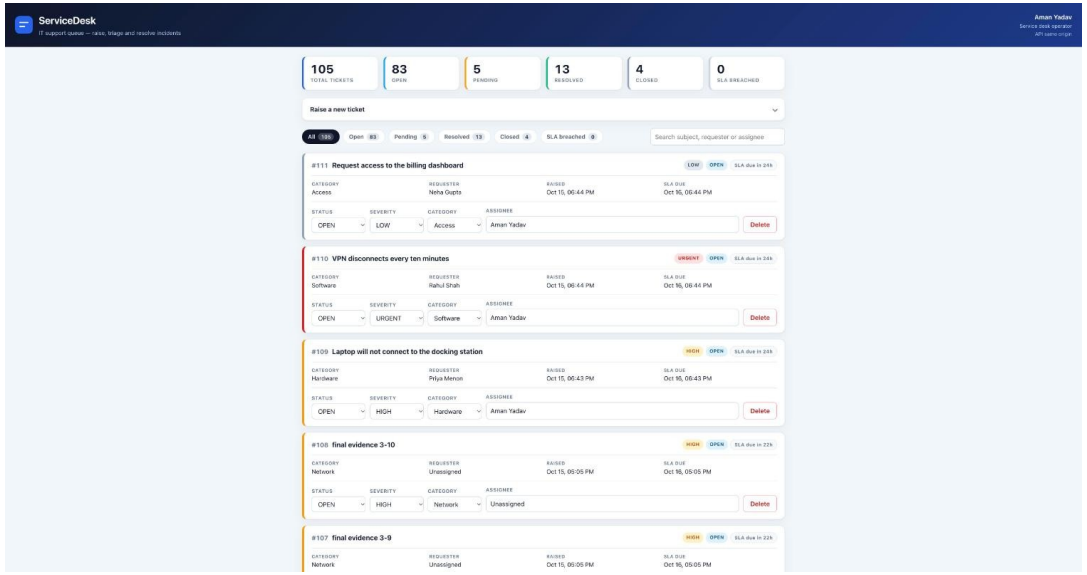
A support team lives and dies by two questions.
What is open right now, and what is about to breach its SLA.

- A ticket is raised against a category, given a severity and a requester
- It is assigned to someone and moves through OPEN, PENDING, RESOLVED, CLOSED
- From creation it carries an SLA deadline twenty-four hours out
- The stats endpoint rolls the whole board into one response, which the console header and the Grafana dashboard both read

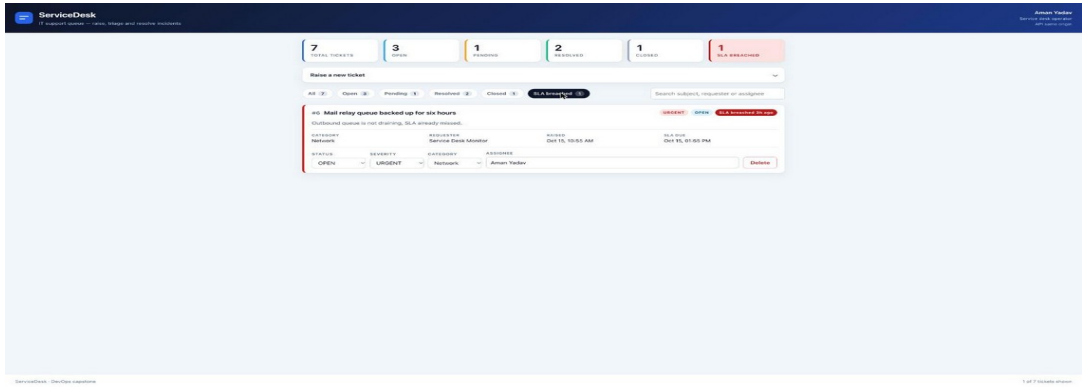
11
API ENDPOINTS

2
TABLES + FK

40
TESTS



The operator console, served through the Kubernetes Ingress



Filter chips carry live counts and narrow the queue without a reload

The data model, and why it has two tables

Ticket

`id · subject · description · severity · status · requester`
`assignee · category_id → categories.id · created_at · sla_due_at`

severity LOW · MEDIUM · HIGH · URGENT
status OPEN · PENDING · RESOLVED · CLOSED

Category

`id · name (unique) · description`

Seeded with Hardware, Software, Access and Network by the second Alembic revision, so a fresh database is usable immediately.

The foreign key is the point, not decoration

A single flat table would have needed one trivial migration. Two related tables forced a real one, with a constraint the database actually enforces — an insert naming a category that does not exist is rejected by PostgreSQL, not by application code.

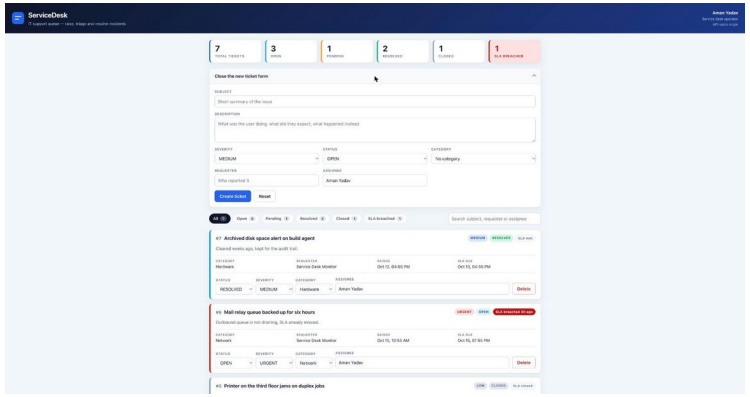
Proven by running the downgrade with a ticket already referencing a seeded category: the first version threw ForeignKeyViolation, which is exactly the bug a migration written without data would hide.

The API surface

GET	`/health`	liveness, no database touched	2xx
GET	`/ready`	readiness, runs a query	2xx
GET	`/metrics`	Prometheus exposition format	2xx
GET	`/api/tickets`	list, newest first	200
POST	`/api/tickets`	create	201
GET	`/api/tickets/{id}`	read one	200 / 404
PUT	`/api/tickets/{id}`	partial update	200 / 404 / 422
DELETE	`/api/tickets/{id}`	remove	204 / 404
GET	`/api/tickets/stats`	counts by status plus SLA breaches	200
GET	`/api/categories`	list	200
POST	`/api/categories`	create, rejects duplicates	201 / 409

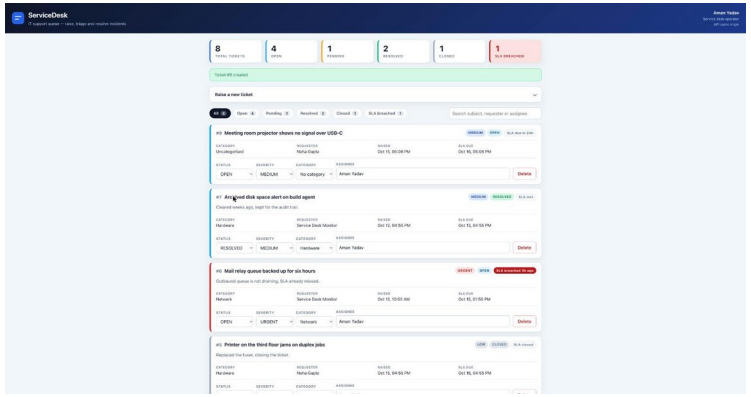
Every status code listed is exercised by the test suite and verified by curl against the running cluster

The console in use



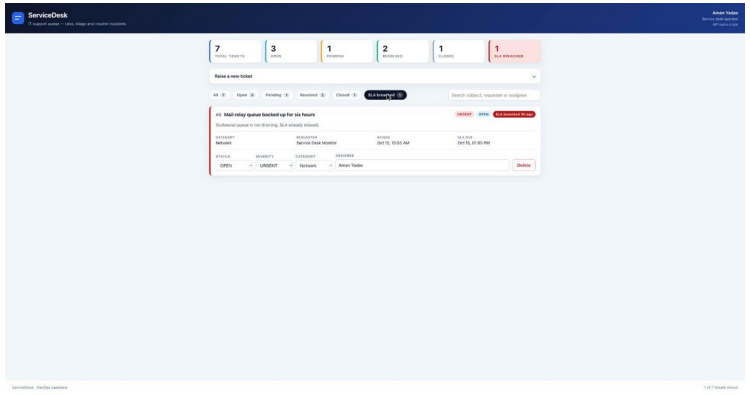
Raising a ticket

Category and severity come from the API, not a hardcoded list



It lands in the queue

Written to PostgreSQL and read back, with the SLA clock already running



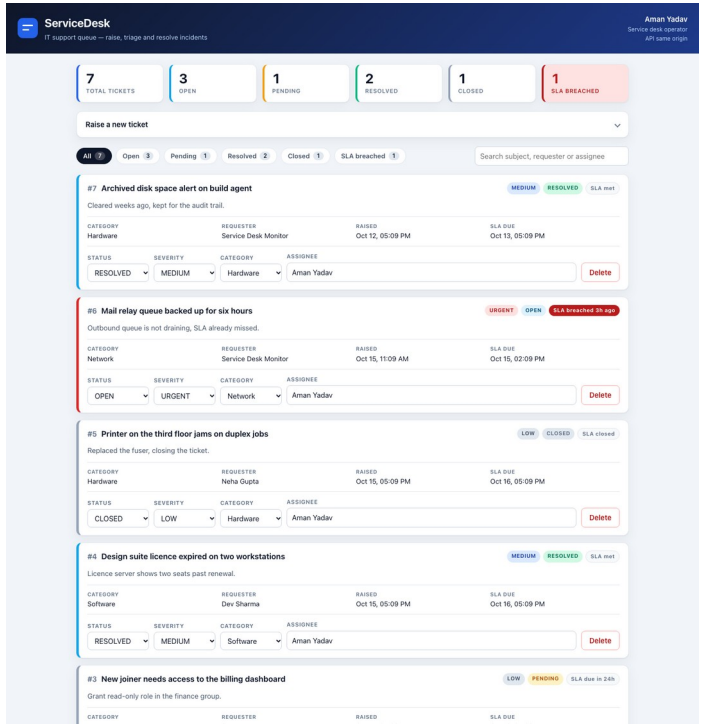
Filtering the board

Chips carry live counts so a breach is one click away

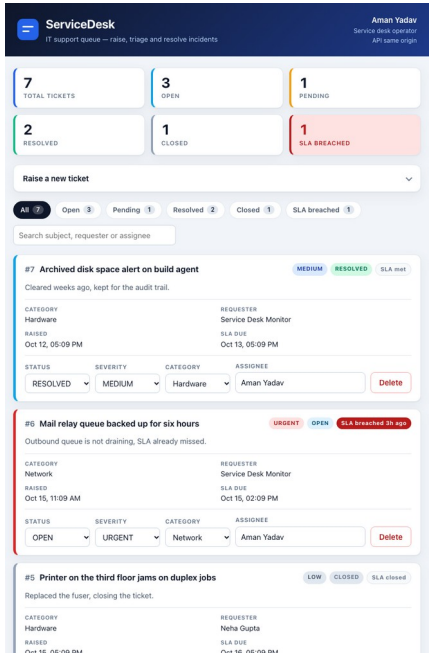
React 19 with TypeScript, built by Vite

Inline editing on every row — status, severity, category and assignee each PUT straight to the API and the stats bar recomputes from the response. Error and success banners cover the failure paths rather than failing silently.

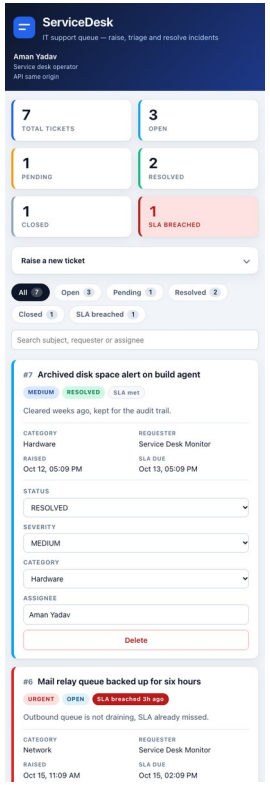
Responsive at three widths



Desktop · 1440px



Tablet · 834px



Phone · 500px

Breakpoints

1000px and 640px.

Below 640 the row grid collapses to stacked fields so nothing overflows horizontally.

The narrow capture is 500px because Chrome on macOS will not render a window narrower than that.

The rubric asks for a responsive, usable UI — these are the same page at three viewport widths

How a change reaches the cluster

1 **Commit** — push to main

→

2 **Test** — pytest gates all

→

3 **Build** — two images

→

4 **Scan** — Trivy, HIGH fails

→

5 **Publish** — GHCR, SHA tag

→

6 **Deploy** — helm upgrade

Nothing in that chain is manual except the final upgrade, and nothing skips a step

Infrastructure

Terraform describes a VPC, four subnets across two availability zones, security groups and an EKS cluster. Applied against an emulator, never real AWS — stated plainly on its own slide.

Orchestration

One Helm chart with three environment overlays. Thirteen revisions covering install, scale, a failed upgrade, rollback and image swaps.

Observability

Prometheus scrapes the backend every fifteen seconds through a ServiceMonitor the chart ships. Grafana reads those series into twenty-three panels.

Tests, and why they cannot touch real data

40
TESTS, ALL PASSING

98%
COVERAGE

5
REQUIRED BY RUBRIC

0.21s
FULL SUITE

What is covered

- Health and readiness, including the database round trip
- Full ticket CRUD and every 404 path
- 409 on a duplicate category name
- 422 on an unknown category_id and on bad enum values
- The stats aggregate, including SLA breach counting

Isolation, proven not claimed

conftest.py resolves the test database before the application is imported.

If that URL ever resolves to the production DATABASE_URL it raises at import rather than running. I verified that refusal path fires.

Every production row was md5-checksummed before and after a full run and came back identical.

Containers

Backend

python:3.12-slim

A builder stage compiles wheels; the runtime copies only the virtualenv. pip, setuptools and wheel are then deleted, which removed four fixable high-severity findings on its own.

Frontend

node build → nginx:1.31-alpine

Node compiles the bundle and only the built assets reach the final image. The compiler, node_modules and source never ship.

Both run as non-root — checked on the published artefact, not the local build

```
$ docker run --rm --entrypoint id ghcr.io/amanyadav7015/servicedesk-backend:5041ecb
  uid=10001(servicedesk) gid=10001(servicedesk) groups=10001(servicedesk)
```

docker compose up --build starts frontend, backend and PostgreSQL together, and the pipeline smoke-tests that same stack on every push — so the compose file cannot silently rot.

The gate is real

A failing test did not just colour a badge red. It prevented an artefact from existing.

Proving the pipeline blocks a bad commit

I pushed a deliberately failing test on commit 759989a

Frontend build (Vite)	success
Backend tests (pytest)	**FAILURE**
Build, scan and publish	skipped
Docker Compose stack	skipped

Skipped, not failed — the needs: graph stopped them before they took a runner

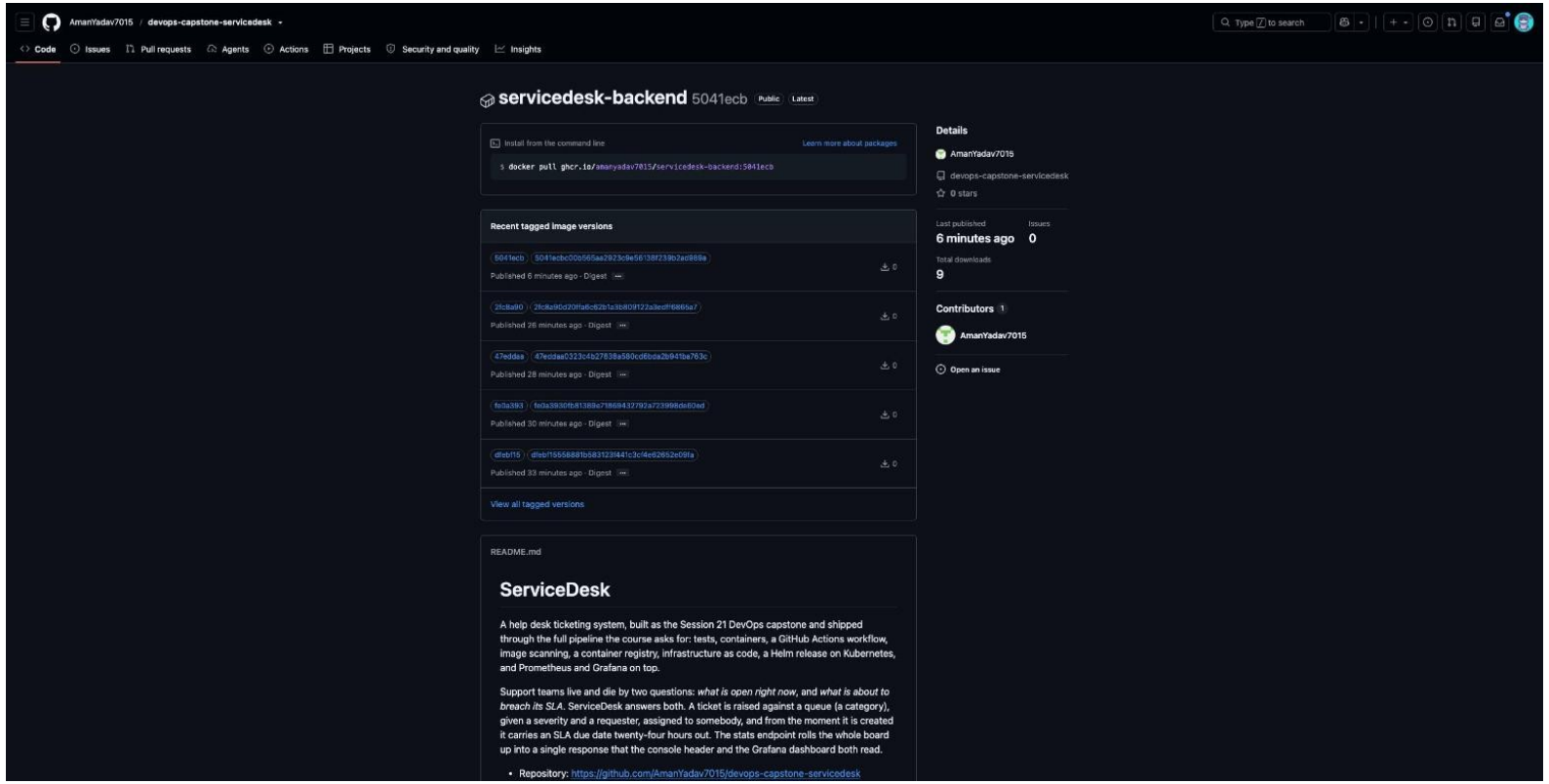
Any running container traces back to the exact source commit that produced it.

The registry settles it

```
red commit   759989a  →  404
green commit d142944  →  200
14 tags, every one a commit SHA
no "latest" tag anywhere
```

I queried the registry directly rather than reading the pipeline log. The red commit produced no artefact at all.

Every image traceable to a commit



The published package: public, SHA-tagged, pullable without authentication

Public

Anyone can pull it; no imagePullSecret is needed in the chart

SHA tags only

Both the full forty-character hash and the short form, never latest

Linked to the repo

The package page carries the README, so provenance is one click away

Security scanning, fixed at source

nginx:1.27-alpine

43

findings
41 high · 2 critical · every one fixable

nginx:1.31-alpine + apk upgrade

0

findings
one minor version, zero suppressions

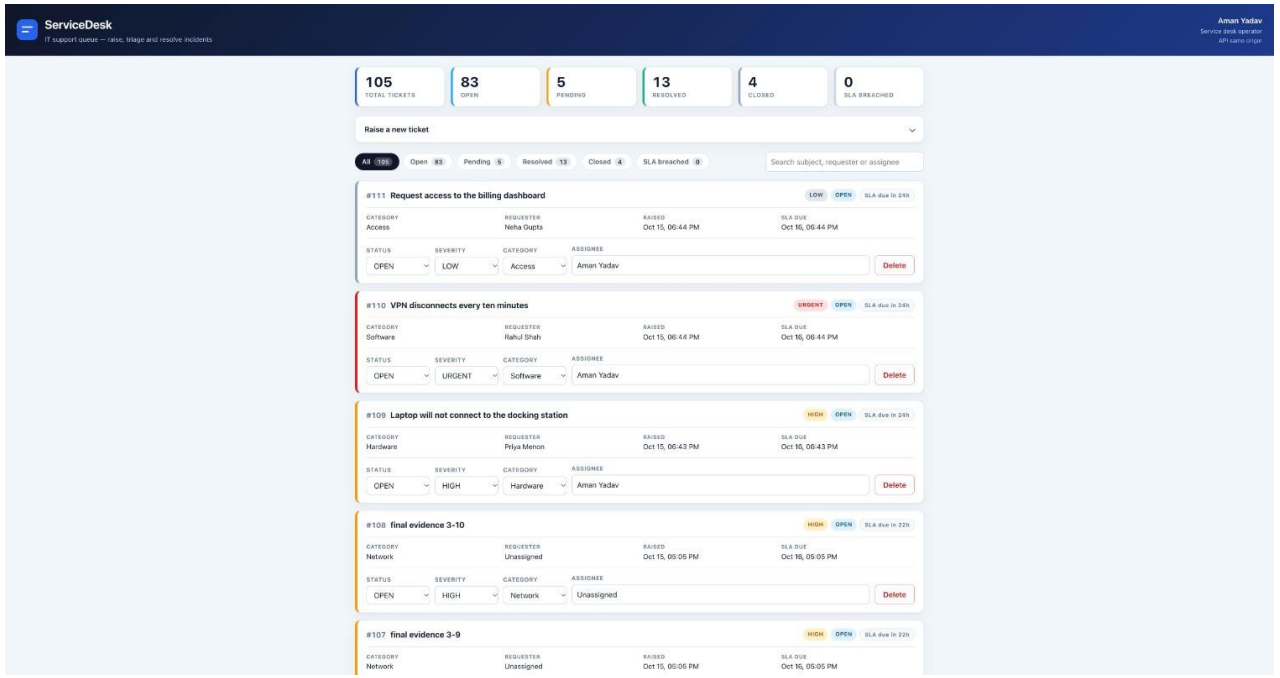
.trivyignore.yaml is empty — and that was a choice, not a default

- The file I inherited suppressed three findings in a web framework. One was already stale.
- The other two were real: the framework was pinned below its fixed release by a second dependency that declared an upper bound.
- Rather than suppress them, I traced the constraint and upgraded the package holding the version down.
- The entire Python section of the report disappeared. Removed at the source, not filtered out of view.

The gate itself runs with --exit-code 1 --severity HIGH,CRITICAL, and an informational pass alongside it hides nothing.

Kubernetes and Helm

5/5	2+2	13	1	4
PODS RUNNING	APP REPLICAS	HELM REVISIONS	INGRESS HOST	CLUSTERIP SVC



Reached at the Ingress hostname, not a port-forward

Proven end to end

```
POST a ticket through the Ingress → 201
delete BOTH backend pods
read the same ticket back → still there
confirmed as a row via psql
```

One Ingress host routes / to the frontend and /api to the backend. maxUnavailable: 0 means two image swaps never dropped a request.

What the chart got wrong, and how I found it

helm lint passed with zero failures. Every defect below was semantic — lint validates syntax, not wiring.

1. **Ingress pointed at a Service that did not exist, on a port nothing listened to** — *would have failed at runtime*
   every /api request would have 503
2. **The frontend could never have started** — *would have failed at runtime*
   its nginx proxies to a host named backend that no Service provided
3. **ServiceMonitor aborted the install where the Prometheus CRDs were absent** — *would have failed on a clean install*
   guarded now with .Capabilities.APIVersions.Has
4. **The database password sat in the Deployment in plain text** — *would have failed on a clean install*
   moved into a Secret and injected by secretKeyRef
5. **Postgres used a rolling strategy on a single read-write volume** — *would have failed on a clean install*
   switched to Recreate so two pods never contend
6. **A startup race between the backend and the database** — *would have failed on a clean install*
   a wait-for-postgres init container closes it

Observability



Twenty-three panels on the application's own instrumentation, not node statistics

2/2

targets up, no scrape errors

9.87 ms

p95 latency under load

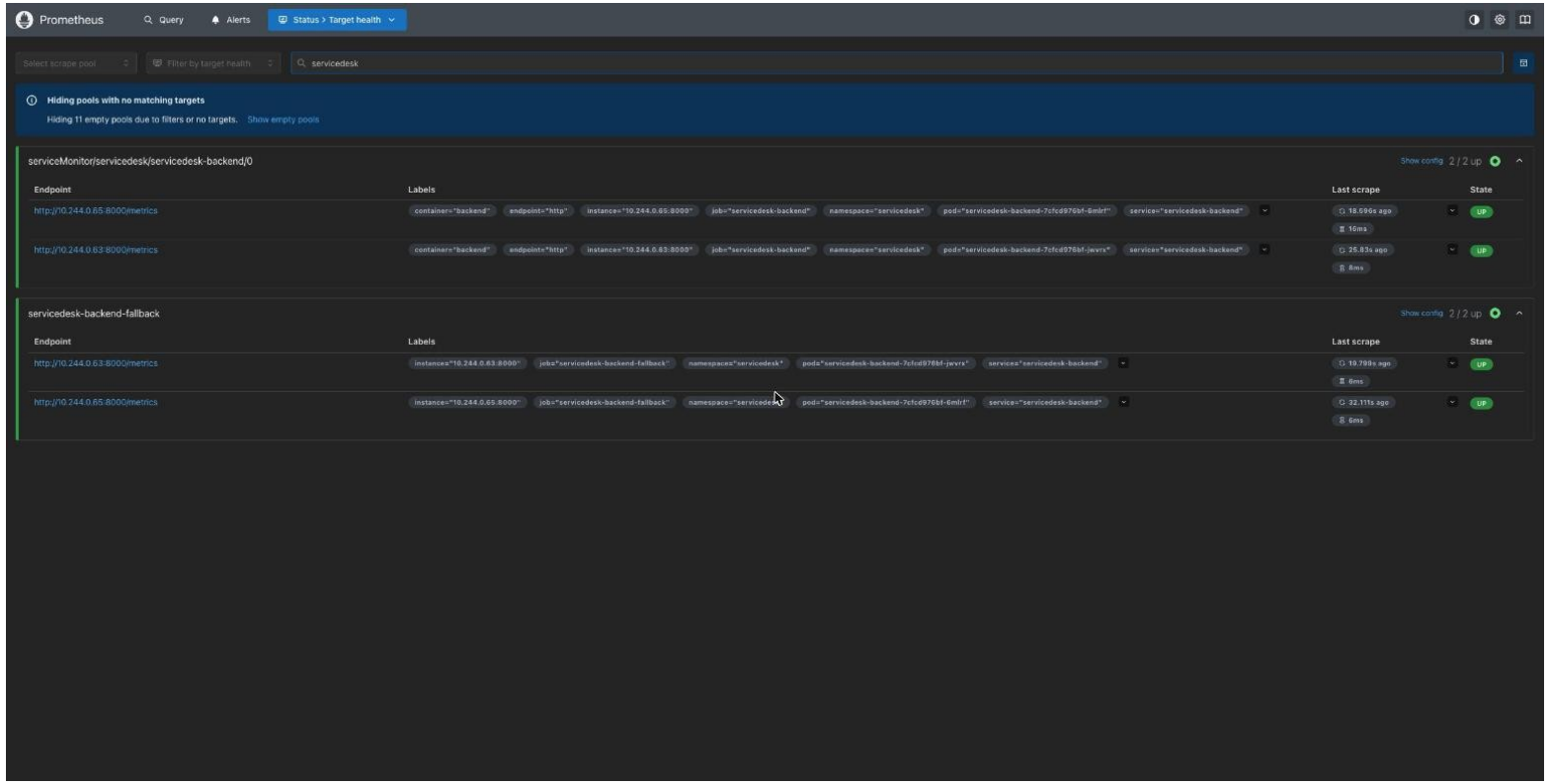
8 + 3

recording rules and alerts

Two bugs in my own queries

A fallback scrape job double-counted every series, and an SLO rule read a histogram bucket that does not exist, so it silently returned nothing.

Where the metrics come from



Both backend pods scraped and up, with an empty last-error on every target

- The chart ships a ServiceMonitor, so the operator discovers pods automatically
- A second fallback job scrapes the same pods by label, which is why each appears twice
- Every query pins job=servicedesk-backend, so nothing double-counts

The instrumentation

/metrics is exposed by the application itself, not an exporter bolted on beside it.

Proven by sending exactly twenty-five requests and watching the counter move by exactly twenty-five.

Infrastructure as code — and what I could not do

Really applied, and really destroyed

Twenty resources — VPC, two public subnets across two availability zones, private subnets, internet gateway, NAT, route tables and security groups. Verified afterwards with the AWS CLI, then destroyed and verified empty.

Planned only, never created

The EKS cluster and its managed node group. A genuine thirty-five resource plan with no errors — but the emulator does not implement the EKS API at all, and there is no AWS account behind this.

I would rather lose the points than imply a cluster exists

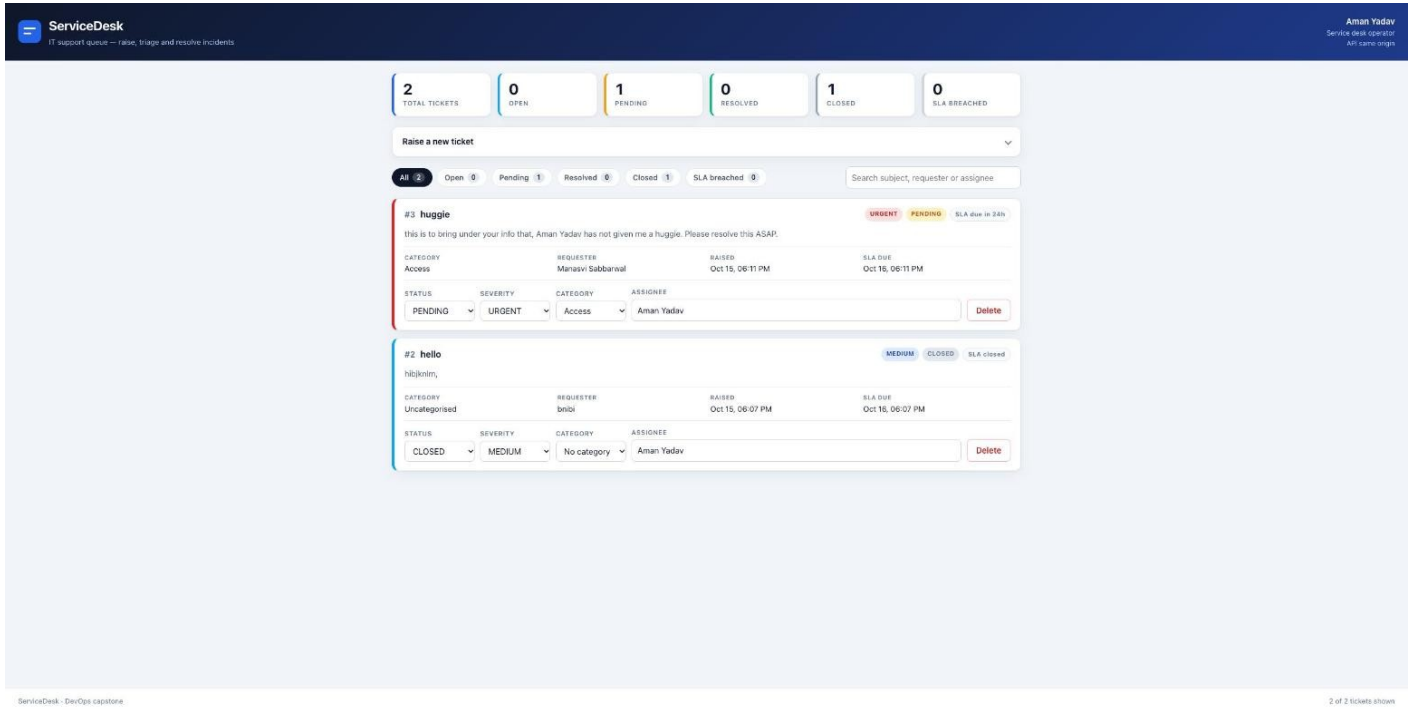
The README states this in its first paragraph. No AWS console screenshot exists and none is claimed.

I also captured the control: with the skip flags off the plan fails with "No valid credential sources found", ~/.aws does not exist, and sts get-caller-identity cannot locate credentials. That is proof the plan was not quietly run against a hidden account.

Three separate probes record the emulator refusing EKS, including Terraform creating the IAM roles successfully and failing only on the cluster call — which shows the HCL is sound and the gap is the emulator.

Realistic score on this module: nine to twelve of fifteen. The four EKS points are unreachable without an account.

Also running on hosted infrastructure



The same images, deployed to a managed platform from the same Dockerfiles

- A managed PostgreSQL instance, with Alembic running on first boot
- Backend and frontend as container services, built from the repository
- Defined in render.yaml so the whole stack is reproducible

Stated plainly in the README

Free tier: the services sleep after fifteen minutes and the managed database is removed after thirty days. The README gives the exact dates and says the Kubernetes deployment is the real deliverable.

Passing your own tests is not the same as being portable

Three bugs that passed CI, passed the security scan, and ran fine in Kubernetes

Three bugs only a stricter host revealed

1 Container up, nothing listening

The stock nginx entrypoint derives a checksum by shelling out to the package manager, which blocks when run as a non-root user. The process stayed alive, so Docker reported it healthy. No HTTP ever answered.

FIX

Removed the script this image replaces anyway

2 exec: Operation not permitted

The binary carried a capability so an unprivileged user could bind port 80. A host setting no_new_privileges refuses to execute a file carrying capabilities. Kubernetes allowed it; the stricter host did not.

FIX

Dropped the capability and moved to port 8080

3 502 with a perfectly healthy backend

The proxy forwarded the browser's Host header upstream, and a variable proxy_pass sends no SNI. A platform routing by Host and terminating TLS could do neither.

FIX

Host $proxy_host and proxy_ssl_server_name on

None is visible when the backend is plain HTTP on the same network — which is exactly why the cluster never surfaced them, and why every fix made the Kubernetes image better too.

Where it stands

github.com/AmanYadav7015/devops-capstone-servicedesk

53
commits

40
tests at 98%

13
helm revisions

0
suppressed CVEs